Quiz #6 (Unit 6 - Extended): 4-Bit Arithmetic & Condition Flags

Total Marks: 150.00

Signed vs. Unsigned Bit Calculation, Flag Generation & Range Validation

Name: **Student ID:** **Score:** / 150.00

> **Instructions:** Answer all 12 questions. Refer to the technique hints to trace 4-bit machine behavior. For flag prompts, write **0** or **1**. For status text, specify **Yes** or **No evidence**. Calculators are not permitted.

PART 1: POSITIVE SUMS & OVERFLOW CONDITIONS

Question 1: Addition Calculation (6 + 6)

Mark 20.00 out of 150.00

[FILL IN THE BLANK - ARITHMETIC TRACE]

> **Technique / Concept Hint:** 4-bit addition: when two positive numbers produce a sum greater than +7, the MSB becomes 1, triggering a positive signed overflow while remaining valid in unsigned representation.

6 + 6: Complete the 4-bit calculation and condition flags for both architectures:

4-Bit Signed-Integer Architecture:

Binary: [ ] + [ ] = [ ] → (in Decimal) 6 + 6 = [ ]

Carry flag = [ ] Borrow flag = [ ] Positive Overflow flag = [ ]

Negative Overflow flag = [ ] Zero flag = [ ]

4-Bit Unsigned-Integer Architecture:

Binary: [ ] + [ ] = [ ] → (in Decimal) 6 + 6 = [ ]

Carry flag = [ ] Borrow flag = [ ] Positive Overflow flag = [ ]

Zero flag = [ ]

Question 2: Signed Overflow Principles

Mark 5.00 out of 150.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Two's complement overflow logic: overflow occurs if and only if two operands of identical sign produce a result of opposite sign (+A + (+B) = -C or -A + (-B) = +C).

Which of the following conditions will trigger the **Positive Overflow flag** in a 4-bit signed architecture? (Select all that apply.)

- ☐ a. Adding two positive numbers whose mathematical sum is strictly greater than 7.
- ☐ b. Adding a positive number and a negative number.
- ☐ c. Computing 5 + 4 in a 4-bit signed register.
- ☐ d. An operation where the carry into the sign bit (C_{in}) does not equal the carry out of the sign bit (C_{out}).

PART 2: MIXED-SIGN ARITHMETIC & INVALID REPRESENTATION

Question 3: Mixed-Sign Addition (-5 + 3)

Mark 20.00 out of 150.00

[FILL IN THE BLANK - ARITHMETIC TRACE]

Technique / Concept Hint: Mixed-sign two's complement addition cannot cause signed overflow; in unsigned architecture, negative numbers do not exist natively and must be evaluated as forced bit patterns.

-5 + 3: Complete the 4-bit calculation and condition flags:

4-Bit Signed-Integer Architecture:

Binary: [ ] + [ ] = [ ] → (in Decimal) −5 + 3 = [ ]

Carry flag = [ ] Borrow flag = [ ] Positive Overflow flag = [ ]

Negative Overflow flag = [ ] Zero flag = [ ]

4-Bit Unsigned-Integer Architecture (if machine forces the bits):

Forced Binary: [ ] + [ ] = [ ]

Unsigned decimal interpretation: [ ] + [ ] = [ ]

Carry flag = [ ] | Zero flag = [ ]

Question 4: Unsigned Register Semantics

Mark 5.00 out of 150.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Register interpretation: raw bits in an unsigned register possess strictly non-negative positional weights (2^3, 2^2, 2^1, 2^0).

Why is the expression –5 + 3 considered inherently invalid in a pure 4-bit unsigned architecture, even if the ALU can be forced to calculate the bit pattern?

- a. Negative numbers cannot exist in an unsigned bit-architecture register because bit 3 has a positive weight (+8) rather than (-8).
- b. Unsigned ALUs lack hardware adders and can only shift bits left or right.
- c. An unsigned register immediately clears itself to `0000` whenever a negative operand is loaded.
- d. The carry-in line is permanently disconnected in unsigned architectures.

PART 3: SUBTRACTION AS ADDITION & BORROW FLAGS

Question 5: Subtraction Calculation (3 - 7)

Mark 25.00 out of 150.00

[FILL IN THE BLANK - ARITHMETIC TRACE]

Technique / Concept Hint: Subtracting a larger number from a smaller number (A < B): signed architecture computes A + (-B) yielding a negative number with Borrow = 1; unsigned architecture wraps around modulo 16 (A - B + 16) with Borrow = 1.

3 – 7: Complete the 4-bit calculation and condition flags:

4-Bit Signed-Integer Architecture (Subtraction as addition: 3 + (–7)):

Binary: [ ] + [ ] = [ ]

Decimal result: 3 – 7 = [ ]

Carry flag = [ ] Borrow flag = [ ] Positive Overflow flag = [ ]

Negative Overflow flag = [ ] Zero flag = [ ]

4-Bit Unsigned-Integer Architecture:

Unsigned result: 3 – 7 = [ ] (in Decimal, wrapped modulo 16)

Carry flag = [ ] Borrow flag = [ ] Zero flag = [ ]

Question 6: Borrow Flag & Modulo Wrap-around

Mark 5.00 out of 150.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Unsigned integer underflow: when A < B, an unsigned subtractor generates a borrow (Borrow = 1) and wraps the result around modulo 2^n.

Which of the following statements regarding the **Borrow flag** and unsigned integer subtraction are **TRUE**? (Select all that apply.)

- [ ] a. When subtracting unsigned integers (A - B), a Borrow is generated if and only if A < B.
- [ ] b. In a 4-bit unsigned register, subtracting 5 from 2 yields 13 due to modulo-16 wrap-around (2 - 5 + 16 = 13).
- [ ] c. In a signed two's complement system, subtracting two positive numbers can never cause a signed overflow.
- [ ] d. Generating a Borrow during an unsigned subtraction indicates that the result is out of the range [0, 15].

PART 4: OPERAND BOUND CHECKS & IDENTITY SUBTRACTION

Question 7: Out-of-Range Operand Tracing (11 - 4)

Mark 15.00 out of 150.00

[FILL IN THE BLANK - ARCHITECTURAL ANALYSIS]

Technique / Concept Hint: Input operand validation: verifying that operands fall within the target architecture's representable limits before operation execution.

11 – 4: Analyze the validity and execution of this operation across both architectures:

4-Bit Signed-Integer Architecture:

11 – 4 is [ ] (valid / invalid) because 11 is a/an [ ]

(positive overflow / negative overflow) in the 4-bit signed architecture.

4-Bit Unsigned-Integer Architecture:

Binary: [ ] – [ ] = [ ]

Decimal result: 11 – 4 = [ ]

Carry flag = [ ] Borrow flag = [ ] Overflow flag = [ ]

Zero flag = [ ]

Question 8: Representable Domain Boundaries

Mark 5.00 out of 150.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: 4-bit range limits: Signed [-8, +7] vs Unsigned [0, 15].

Why can the operation 12 − 5 be performed legitimately in a 4-bit unsigned architecture, but fails before execution in a 4-bit signed architecture?

- ◯ a. Because 12 is within the valid unsigned range [0, 15], but exceeds the maximum signed 4-bit upper bound of +7.
- ◯ b. Because signed architectures cannot subtract odd numbers like 5.
- ◯ c. Because the carry flag is permanently held high during signed subtractions.
- ◯ d. Because 12 in binary starts with a 1, which the ALU unconditionally rejects as an illegal opcode.

Question 9: Identity Subtraction (4 - 4)

Mark 25.00 out of 150.00

[FILL IN THE BLANK - ARITHMETIC TRACE]

Technique / Concept Hint: Identity subtraction A - A: result is always 0, setting the Zero flag (Z = 1); when computed as A + (~A + 1), the adder produces a carry-out (C = 1) with no overflow.

4 – 4: Complete the 4-bit calculation and condition flags:

4-Bit Signed-Integer Architecture (Addition with two's complement: 4 + (–4)):

Binary: [ ] + [ ] = [ ]

Carry flag = [ ] Borrow flag = [ ] Positive Overflow flag = [ ]

Zero flag = [ ]

4-Bit Unsigned-Integer Architecture (Direct A - B procedure):

Binary: [ ] – [ ] = [ ]

Carry flag = [ ] Borrow flag = [ ] Zero flag = [ ]

Question 10: Zero Flag & Identity Subtraction Conditions

Mark 5.00 out of 150.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Zero flag (Z) status and carry generation when subtracting identical non-zero values.

Which of the following statements are **TRUE** when a CPU executes identity subtraction on identical non-zero numbers (A - A)? (Select all that apply.)

- ☐ a. The Zero flag (Z) is set to 1 because every output bit in the result register is 0.
- ☐ b. When implemented via adder hardware (A + ~A + 1), a carry-out of the MSB (C = 1) is generated.
- ☐ c. The Borrow flag is set to 1 because A is equal to A.
- ☐ d. The Overflow flag is 0 because the result (0) is representable in both signed and unsigned formats.

PART 5: UNDERFLOW WRAPPING & INVERTED CARRY SEMANTICS

Question 11: Negative Result Subtraction (2 - 6)

Mark 20.00 out of 150.00

[FILL IN THE BLANK - ARITHMETIC TRACE]

Technique / Concept Hint: Subtracting 2 - 6: signed system produces -4 (1100_2); unsigned system wraps to 12 (1100_2) with Borrow = 1.

2 – 6: Complete the 4-bit calculation and condition flags:

4-Bit Signed-Integer Architecture:

Binary: [ ] + [ ] = [ ] → Decimal result: [ ]

Borrow flag = [ ] Overflow flag = [ ] Zero flag = [ ]

4-Bit Unsigned-Integer Architecture:

Binary: [ ] – [ ] = [ ] → Modulo 16 Decimal result: [ ]

Borrow flag = [ ] Zero flag = [ ]

Question 12: Inverted Carry as Borrow in Hardware

Mark 5.00 out of 150.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Adder-subtractor hardware relationship: when evaluating A - B as A + (~B + 1), a carry-out of 1 signifies NO borrow, whereas a carry-out of 0 signifies a BORROW.

When an ALU executes subtraction (A - B) by computing A + (~B + 1) on an adder circuit, how is the **Borrow flag** derived from the adder's Carry-out (C_{out})?

- ◯ a. Borrow = NOT(C_{out}) (i.e., Carry-out = 0 means a Borrow occurred, Carry-out = 1 means No Borrow).
- ◯ b. Borrow = C_{out} directly without inversion.
- ◯ c. Borrow = C_{out} AND Zero flag.
- ◯ d. Borrow is read exclusively from the sign bit of the second operand.

Instructor Answer Key & Detailed Marking Scheme

Maximum Marks: 150.00

Quiz #6 (Unit 6 - Extended) - Step-by-Step Flag Logic & Verifications

---

Question 1 Solution: 6 + 6 Execution (20.00 Marks)

Signed: [0110] + [0110] = [1100] → Decimal: [-4]
Flags: Carry = [0], Borrow = [0], Pos Overflow = [1], Neg Overflow = [0], Zero = [0]
Unsigned: [0110] + [0110] = [1100] → Decimal: [12]
Flags: Carry = [0], Borrow = [0], Pos Overflow = [0], Zero = [0]

Explanation: 6 = 0110_2. $0110_2 + 0110_2 = 1100_2$. In 4-bit signed, 1100_2 = -4. Adding two positives produced a negative result because +12 > 7, setting Positive Overflow = 1. In unsigned, 1100_2 = 12, which is ≤ 15, producing no carry or overflow.

Question 2 Solution: Signed Overflow Conditions (5.00 Marks)

Correct Answers: a, c, d

Explanation:

- **a is TRUE:** The maximum positive 4-bit signed integer is +7; any positive sum > 7 overflows.
- **b is FALSE:** Adding operands of opposite signs can never cause overflow.
- **c is TRUE:** 5 + 4 = 9 > 7 produces $0101_2 + 0100_2 = 1001_2$ (-7), triggering positive overflow.
- **d is TRUE:** By definition of hardware overflow logic, $V = C_{in} \oplus C_{out}$.

Question 3 Solution: -5 + 3 Execution (20.00 Marks)

Signed: [1011] + [0011] = [1110] → Decimal: [-2]
Flags: Carry = [0], Borrow = [0], Pos Overflow = [0], Neg Overflow = [0], Zero = [0]
Unsigned Forced: [1011] + [0011] = [1110] → Unsigned Decimal: [11] + [3] = [14]
Flags: Carry = [0], Zero = [0]

Explanation: -5 in 4-bit two's complement is 1011_2, and +3 is 0011_2. $1011_2 + 0011_2 = 1110_2$ = -2. No overflow occurs because operands have opposite signs. Unsigned registers do not natively represent -5; forced bits treat 1011_2 as 11, yielding 11 + 3 = 14 (1110_2).

Question 4 Solution: Unsigned Register Semantics (5.00 Marks)

Correct Answer: a

Explanation: In an unsigned architecture, bit 3 represents $+2^3$ = +8, making negative quantities strictly outside the register's representable domain.

Question 5 Solution: 3 - 7 Execution (25.00 Marks)

Signed: [0011] + [1001] = [1100] → Decimal: [-4]
Flags: Carry = [0], Borrow = [1], Pos Overflow = [0], Neg Overflow = [0], Zero = [0]
Unsigned: 3 - 7 = [12] (modulo 16)
Flags: Carry = [0], Borrow = [1], Zero = [0]

Explanation: 3 - 7 = 3 + (-7) = 0011_2 + 1001_2 = 1100_2. Signed value is -4. In unsigned arithmetic, subtracting a larger number from a smaller number produces a borrow (Borrow = 1) and wraps around modulo 16: 3 - 7 + 16 = 12 (1100_2).

Question 6 Solution: Borrow & Modulo Wrap-around (5.00 Marks)

Correct Answers: a, b, c, d

Explanation: All statements are true: Borrow occurs when A < B in unsigned math, results wrap modulo 2^n (2 - 5 ≡ 13 mod 16), subtracting two positive numbers cannot cause signed overflow (as signs are opposite during subtraction), and borrow confirms an out-of-range unsigned condition.

Question 7 Solution: 11 - 4 Execution (15.00 Marks)

Signed: [invalid] | [positive overflow]
Unsigned: [1011] - [0100] = [0111] → Decimal: [7]
Flags: Carry = [0], Borrow = [0], Overflow = [0], Zero = [0]

Explanation: Decimal 11 cannot be loaded into a 4-bit signed register because the maximum positive limit is +7; 11 is an input positive overflow. In unsigned, 11 is 1011_2, and 1011_2 - 0100_2 = 0111_2 = 7, with all flag conditions clear.

Question 8 Solution: Domain Boundaries (5.00 Marks)

Correct Answer: a

Explanation: The valid range for 4-bit unsigned is [0, 15], accommodating 12. The valid range for 4-bit signed is [-8, +7], meaning 12 cannot be represented without immediate overflow.

Question 9 Solution: 4 - 4 Identity Subtraction (25.00 Marks)

Signed: [0100] + [1100] = [0000]
Flags: Carry = [1], Borrow = [0], Pos Overflow = [0], Zero = [1]
Unsigned: [0100] - [0100] = [0000]
Flags: Carry = [0], Borrow = [0], Zero = [1]

Explanation: Evaluating 4 + (-4) = 0100_2 + 1100_2 = 10000_2 → 0000_2. The 5th bit sets Carry = 1. Because the result register contains all zeros, Zero flag = 1. Borrow = 0 because 4 ≥ 4.

Question 10 Solution: Zero Flag Conditions (5.00 Marks)

Correct Answers: a, b, d

Explanation:

- **a is TRUE:** The Zero flag asserts whenever all output register bits evaluate to zero.
- **b is TRUE:** Adding A to its two's complement (~A + 1) always produces an adder carry-out of 1 for any non-zero A.
- **c is FALSE:** Borrow is 0 because A is not less than A.
- **d is TRUE:** Zero is representable in both systems without overflow.

Question 11 Solution: 2 - 6 Execution (20.00 Marks)

Signed: [0010] + [1010] = [1100] → Decimal: [-4]
Flags: Borrow = [1], Overflow = [0], Zero = [0]

Unsigned: [0010] - [0110] = [1100] → Modulo 16 Decimal: [12]
Flags: Borrow = [1], Zero = [0]

Explanation: 2 - 6 = 2 + (-6) = $0010_2 + 1010_2 = 1100_2$ = -4. In unsigned, 2 - 6 = -4 ≡ 12 mod 16 (1100_2), generating a Borrow of 1 because 2 < 6.

Question 12 Solution: Inverted Carry Mechanics (5.00 Marks)

Correct Answer: a

Explanation: In adder-subtractor circuits, A - B is computed as A + ~B + 1. A carry-out of 1 indicates that the result did not require a borrow, whereas a carry-out of 0 indicates that a borrow was required (Borrow = NOT(C_{out})).
